

|      | Опис граматики за допомогою РБНФ | Опис граматика<br>(правила)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | LL2-нотація для правил<br>граматикиОпис граматика |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Код для перевірки граматики<br>заданої за допомогою РБНФ | Дані для перевірки перевірки граматики за допомогою LL2-аналізатора                                          |  |
|------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--|
| РБНФ |                                  | G = (N, T, P, S)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                   | Код для перевірки РБНФ                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                          | УВАГА: при копіюванні зважайте, щоб у кожному рядку після символу «\»<br>не містилось жодних інших символів. |  |
|      |                                  | N = {<br>labeled_point<br>ident<br>goto_label<br>program_name<br>value_type<br>array_specify<br>declaration_element<br>array_specify_optional<br>other_declaration_ident<br>declaration<br>other_declaration_ident__iteration<br>index_action<br>unary_operator<br>unary_operation<br>binary_operator<br>binary_action<br>left_expression<br>group_expression<br>value<br>index_action__optional<br>expression<br>binary_action__iteration<br>expression_or_bind_left_to_right<br>bind_to_right<br>bind_to_right__optional<br>if_expression<br>body_for_true<br>false_cond_block_without_else<br>body_for_false<br>cond_block<br>false_cond_block_without_else__iteration<br>body_for_false__optional<br>cond_block__with_optional_bind<br>cycle_begin_expression<br>cycle_end_expression<br>cycle_counter<br>cycle_counter_lr_init<br>cycle_counter_init<br>cycle_counter_last_value<br>cycle_body<br>forto_direction<br>forto_cycle<br>continue_while<br>break_while<br>statement_in_while_and_if_body<br>statement<br>block_statements_in_while_and_if_body<br>statement_in_while_and_if_body__iteration<br>while_cycle_head_expression<br>while_cycle<br>repeat_until_cycle_cond<br>repeat_until_cycle<br>statements__or__block_statements<br>block_statements<br>input_rule<br>argument_for_input<br>output_rule<br>statement__iteration<br>expression__optional<br>program_rule<br>declaration__optional<br>unsigned_value<br>value<br>sign_optional<br>sign<br>ident<br>sign_plus<br>sign_minus<br>}<br><br>T = {<br>".",<br>"GOTO",<br>"INTEGER16",<br>" ",<br>"/",<br>"NOT",<br>"AND",<br>"OR",<br>"==",<br>"!=",<br>"<=",<br>">=",<br>"<",<br>">",<br>"+",<br>"-",<br>"*",<br>"/",<br>"DIV",<br>"MOD",<br>"(",<br>")",<br>":",<br>";",<br>"ELSE",<br>"IF",<br>"DO",<br>"FOR",<br>"TO",<br>"DOWNT",<br>"WHILE",<br>"CONTINUE",<br>"BREAK",<br>"EXIT",<br>"REPEAT",<br>} |                                                   | N = {<br>labeled_point<br>ident<br>goto_label<br>program_name<br>value_type<br>array_specify<br>declaration_element<br>array_specify_optional<br>other_declaration_ident<br>declaration<br>other_declaration_ident__iteration<br>index_action<br>unary_operator<br>unary_operation<br>binary_operator<br>binary_action<br>left_expression<br>group_expression<br>value<br>index_action__optional<br>expression<br>binary_action__iteration<br>expression_or_bind_left_to_right<br>bind_to_right<br>bind_to_right__optional<br>if_expression<br>body_for_true<br>false_cond_block_without_else<br>body_for_false<br>cond_block<br>false_cond_block_without_else__iteration<br>body_for_false__optional<br>cond_block__with_optional_bind<br>cycle_begin_expression<br>cycle_end_expression<br>cycle_counter<br>cycle_counter_lr_init<br>cycle_counter_init<br>cycle_counter_last_value<br>cycle_body<br>forto_direction<br>forto_cycle<br>continue_while<br>break_while<br>statement_in_while_and_if_body<br>statement<br>block_statements_in_while_and_if_body<br>statement_in_while_and_if_body__iteration<br>while_cycle_head_expression<br>while_cycle<br>repeat_until_cycle_cond<br>repeat_until_cycle<br>statements__or__block_statements<br>block_statements<br>input_rule<br>argument_for_input<br>output_rule<br>statement__iteration<br>expression__optional<br>program_rule<br>declaration__optional<br>unsigned_value<br>value<br>sign_optional<br>sign<br>ident<br>sign_plus<br>sign_minus<br>}<br><br>T = {<br>".",<br>",",<br>"GOTO",<br>"INTEGER16",<br>" ",<br>"/",<br>"NOT",<br>"AND",<br>"OR",<br>"==",<br>"!=",<br>"<=",<br>">=",<br>"<",<br>">",<br>"+",<br>"-",<br>"*",<br>"/",<br>"DIV",<br>"MOD",<br>"(",<br>")",<br>":",<br>";",<br>"ELSE",<br>"IF",<br>"DO",<br>"FOR",<br>"TO",<br>"DOWNT",<br>"WHILE",<br>"CONTINUE",<br>"BREAK",<br>"EXIT",<br>"REPEAT",<br>} |                                                          |                                                                                                              |  |

|  |  |                                                                                                                                                                 |  |                                                                                                                                                                 |  |                                                                                                                                                                 |
|--|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  | <pre>"UNTIL", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "{", "}", "[", "]", "(", ")", "=", "+", "ident_terminal", "unsigned_value_terminal" }</pre> |  | <pre>"UNTIL", "GET", "PUT", "NAME", "BODY", "DATA", "BEGIN", "END", "{", "}", "[", "]", "(", ")", "=", "+", "ident_terminal", "unsigned_value_terminal" }</pre> |  |                                                                                                                                                                 |
|  |  |                                                                                                                                                                 |  |                                                                                                                                                                 |  |                                                                                                                                                                 |
|  |  |                                                                                                                                                                 |  | tokenCOLON = ":" >> BOUNDARIES;                                                                                                                                 |  | <pre>#define T_COLON_0 ":" #define T_COLON_1 "" #define T_COLON_2 "" #define T_COLON_3 ""</pre>                                                                 |
|  |  |                                                                                                                                                                 |  | tokenGOTO = "GOTO" >> STRICT_BOUNDARIES;                                                                                                                        |  | <pre>#define T_GOTO_0 "GOTO" #define T_GOTO_1 "" #define T_GOTO_2 "" #define T_GOTO_3 ""</pre>                                                                  |
|  |  |                                                                                                                                                                 |  | tokenINTEGER16 = "INTEGER16" >> STRICT_BOUNDARIES;                                                                                                              |  | <pre>#define T_DATA_TYPE_0 "INTEGER16" #define T_DATA_TYPE_1 "" #define T_DATA_TYPE_2 "" #define T_DATA_TYPE_3 ""</pre>                                         |
|  |  |                                                                                                                                                                 |  | tokenCOMMA = "," >> BOUNDARIES;                                                                                                                                 |  | <pre>#define T_COMA_0 "," #define T_COMA_1 "" #define T_COMA_2 "" #define T_COMA_3 ""</pre>                                                                     |
|  |  |                                                                                                                                                                 |  |                                                                                                                                                                 |  | <pre>#define T_BITWISE_NOT_0 "~" #define T_BITWISE_NOT_1 "" #define T_BITWISE_NOT_2 "" #define T_BITWISE_NOT_3 ""</pre>                                         |
|  |  |                                                                                                                                                                 |  | tokenNOT = "NOT" >> STRICT_BOUNDARIES;                                                                                                                          |  | <pre>#define T_NOT_0 "NOT" #define T_NOT_1 "" #define T_NOT_2 "" #define T_NOT_3 ""</pre>                                                                       |
|  |  |                                                                                                                                                                 |  |                                                                                                                                                                 |  | <pre>#define T_BITWISE_AND_0 "&amp;" #define T_BITWISE_AND_1 "" #define T_BITWISE_AND_2 "" #define T_BITWISE_AND_3 ""</pre>                                     |
|  |  |                                                                                                                                                                 |  | tokenAND = "AND" >> STRICT_BOUNDARIES;                                                                                                                          |  | <pre>#define T_AND_0 "AND" #define T_AND_1 "" #define T_AND_2 "" #define T_AND_3 ""</pre>                                                                       |
|  |  |                                                                                                                                                                 |  |                                                                                                                                                                 |  | <pre>#define T_BITWISE_OR_0 " " #define T_BITWISE_OR_1 "" #define T_BITWISE_OR_2 "" #define T_BITWISE_OR_3 ""</pre>                                             |
|  |  |                                                                                                                                                                 |  | tokenOR = "OR" >> STRICT_BOUNDARIES;                                                                                                                            |  | <pre>#define T_OR_0 "OR" #define T_OR_1 "" #define T_OR_2 "" #define T_OR_3 ""</pre>                                                                            |
|  |  |                                                                                                                                                                 |  | tokenEQUAL = "==" >> BOUNDARIES;                                                                                                                                |  | <pre>#define T_EQUAL_0 "==" #define T_EQUAL_1 "" #define T_EQUAL_2 "" #define T_EQUAL_3 ""</pre>                                                                |
|  |  |                                                                                                                                                                 |  | tokenNOTEQUAL = "!=" >> BOUNDARIES;                                                                                                                             |  | <pre>#define T_NOT_EQUAL_0 "!=" #define T_NOT_EQUAL_1 "" #define T_NOT_EQUAL_2 "" #define T_NOT_EQUAL_3 ""</pre>                                                |
|  |  |                                                                                                                                                                 |  | tokenLESSOREQUAL = "<=" >> BOUNDARIES;                                                                                                                          |  | <pre>#define T_LESS_OR_EQUAL_0 "&lt;=" #define T_LESS_OR_EQUAL_1 "" #define T_LESS_OR_EQUAL_2 "" #define T_LESS_OR_EQUAL_3 ""</pre>                             |
|  |  |                                                                                                                                                                 |  | tokenGREATEROREQUAL = ">=" >> BOUNDARIES;                                                                                                                       |  | <pre>#define T_GREATER_OR_EQUAL_0 "&gt;=" #define T_GREATER_OR_EQUAL_1 "" #define T_GREATER_OR_EQUAL_2 "" #define T_GREATER_OR_EQUAL_3 ""</pre>                 |
|  |  |                                                                                                                                                                 |  | tokenLESS = "<" >> BOUNDARIES;                                                                                                                                  |  | <pre>#define T_LESS_0 "&lt;" #define T_LESS_1 "" #define T_LESS_2 "" #define T_LESS_3 ""</pre>                                                                  |
|  |  |                                                                                                                                                                 |  | tokenGREATER = ">" >> BOUNDARIES;                                                                                                                               |  | <pre>#define T_GREATER_0 "&gt;" #define T_GREATER_1 "" #define T_GREATER_2 "" #define T_GREATER_3 ""</pre>                                                      |
|  |  |                                                                                                                                                                 |  | tokenPLUS = "+" >> BOUNDARIES;                                                                                                                                  |  | <pre>#define T_ADD_0 "+" #define T_ADD_1 "" #define T_ADD_2 "" #define T_ADD_3 ""</pre>                                                                         |
|  |  |                                                                                                                                                                 |  | tokenMINUS = "-" >> BOUNDARIES;                                                                                                                                 |  | <pre>#define T_SUB_0 "-" #define T_SUB_1 "" #define T_SUB_2 "" #define T_SUB_3 ""</pre>                                                                         |
|  |  |                                                                                                                                                                 |  | tokenMUL = "*" >> BOUNDARIES;                                                                                                                                   |  | <pre>#define T_MUL_0 "*" #define T_MUL_1 "" #define T_MUL_2 "" #define T_MUL_3 ""</pre>                                                                         |
|  |  |                                                                                                                                                                 |  | tokenDIV = "DIV" >> STRICT_BOUNDARIES;                                                                                                                          |  | <pre>#define T_DIV_0 "DIV" #define T_DIV_1 "" #define T_DIV_2 "" #define T_DIV_3 ""</pre>                                                                       |
|  |  |                                                                                                                                                                 |  | tokenMOD = "MOD" >> STRICT_BOUNDARIES;                                                                                                                          |  | <pre>#define T_MOD_0 "MOD" #define T_MOD_1 "" #define T_MOD_2 "" #define T_MOD_3 ""</pre>                                                                       |
|  |  |                                                                                                                                                                 |  | tokenGROUPEXPRESSIONBEGIN = "(" >> BOUNDARIES;                                                                                                                  |  | <pre>#define T_BEGIN GROUPEXPRESSION_0 "(" #define T_BEGIN GROUPEXPRESSION_1 "" #define T_BEGIN GROUPEXPRESSION_2 "" #define T_BEGIN GROUPEXPRESSION_3 ""</pre> |

|  |  |  |  |                                                  |  |                                                                                                                                                       |
|--|--|--|--|--------------------------------------------------|--|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  |  |  | tokenGROUPEXPRESSIONEND = "]" >> BOUNDARIES;     |  | #define T_END_GROUPEXPRESSION_0 "]"<br>#define T_END_GROUPEXPRESSION_1 ""<br>#define T_END_GROUPEXPRESSION_2 ""<br>#define T_END_GROUPEXPRESSION_3 "" |
|  |  |  |  | tokenLRBIND = "=:;" >> BOUNDARIES;               |  | #define T_LRBIND_0 "=:;"<br>#define T_LRBIND_1 ""<br>#define T_LRBIND_2 ""<br>#define T_LRBIND_3 ""                                                   |
|  |  |  |  |                                                  |  | #define T_THEN_BLOCK_0 "{"<br>#define T_THEN_BLOCK_1 ""<br>#define T_THEN_BLOCK_2 ""<br>#define T_THEN_BLOCK_3 ""                                     |
|  |  |  |  | tokenELSE = "ELSE" >> STRICT_BOUNDARIES;         |  | #define T_ELSE_BLOCK_0 "ELSE"<br>#define T_ELSE_BLOCK_1 "{"<br>#define T_ELSE_BLOCK_2 ""<br>#define T_ELSE_BLOCK_3 ""                                 |
|  |  |  |  | tokenIF = "IF" >> STRICT_BOUNDARIES;             |  | #define T_IF_0 "IF"<br>#define T_IF_1 ""<br>#define T_IF_2 ""<br>#define T_IF_3 ""                                                                    |
|  |  |  |  |                                                  |  | #define T_ELSE_IF_0 "ELSE"<br>#define T_ELSE_IF_1 "IF"<br>#define T_ELSE_IF_2 ""<br>#define T_ELSE_IF_3 ""                                            |
|  |  |  |  | tokenDO = "DO" >> STRICT_BOUNDARIES;             |  | #define T_DO_0 "DO"<br>#define T_DO_1 ""<br>#define T_DO_2 ""<br>#define T_DO_3 ""                                                                    |
|  |  |  |  | tokenFOR = "FOR" >> STRICT_BOUNDARIES;           |  | #define T_FOR_0 "FOR"<br>#define T_FOR_1 ""<br>#define T_FOR_2 ""<br>#define T_FOR_3 ""                                                               |
|  |  |  |  | tokenTO = "TO" >> STRICT_BOUNDARIES;             |  | #define T_TO_0 "TO"<br>#define T_TO_1 ""<br>#define T_TO_2 ""<br>#define T_TO_3 ""                                                                    |
|  |  |  |  | tokenDOWNT0 = "DOWNT0" >> STRICT_BOUNDARIES;     |  | #define T_DOWNT0_0 "DOWNT0"<br>#define T_DOWNT0_1 ""<br>#define T_DOWNT0_2 ""<br>#define T_DOWNT0_3 ""                                                |
|  |  |  |  | tokenWHILE = "WHILE" >> STRICT_BOUNDARIES;       |  | #define T_WHILE_0 "WHILE"<br>#define T_WHILE_1 ""<br>#define T_WHILE_2 ""<br>#define T_WHILE_3 ""                                                     |
|  |  |  |  | tokenCONTINUE = "CONTINUE" >> STRICT_BOUNDARIES; |  | #define T_CONTINUE_WHILE_0 "CONTINUE"<br>#define T_CONTINUE_WHILE_1 ""<br>#define T_CONTINUE_WHILE_2 ""<br>#define T_CONTINUE_WHILE_3 ""              |
|  |  |  |  | tokenBREAK = "BREAK" >> STRICT_BOUNDARIES;       |  | #define T_EXIT_WHILE_0 "BREAK"<br>#define T_EXIT_WHILE_1 ""<br>#define T_EXIT_WHILE_2 ""<br>#define T_EXIT_WHILE_3 ""                                 |
|  |  |  |  | tokenEXIT = "EXIT" >> STRICT_BOUNDARIES;         |  | #define T_EXIT_0 "EXIT"<br>#define T_EXIT_1 ""<br>#define T_EXIT_2 ""<br>#define T_EXIT_3 ""                                                          |
|  |  |  |  | tokenREPEAT = "REPEAT" >> STRICT_BOUNDARIES;     |  | #define T_REPEAT_0 "REPEAT"<br>#define T_REPEAT_1 ""<br>#define T_REPEAT_2 ""<br>#define T_REPEAT_3 ""                                                |
|  |  |  |  | tokenUNTIL = "UNTIL" >> STRICT_BOUNDARIES;       |  | #define T_UNTIL_0 "UNTIL"<br>#define T_UNTIL_1 ""<br>#define T_UNTIL_2 ""<br>#define T_UNTIL_3 ""                                                     |
|  |  |  |  | tokenGET = "GET" >> STRICT_BOUNDARIES;           |  | #define T_INPUT_0 "GET"<br>#define T_INPUT_1 ""<br>#define T_INPUT_2 ""<br>#define T_INPUT_3 ""                                                       |
|  |  |  |  | tokenPUT = "PUT" >> STRICT_BOUNDARIES;           |  | #define T_OUTPUT_0 "PUT"<br>#define T_OUTPUT_1 ""<br>#define T_OUTPUT_2 ""<br>#define T_OUTPUT_3 ""                                                   |
|  |  |  |  | tokenNAME = "NAME" >> STRICT_BOUNDARIES;         |  | #define T_NAME_0 "NAME"<br>#define T_NAME_1 ""<br>#define T_NAME_2 ""<br>#define T_NAME_3 ""                                                          |
|  |  |  |  | tokenBODY = "BODY" >> STRICT_BOUNDARIES;         |  | #define T_BODY_0 "BODY"<br>#define T_BODY_1 ""<br>#define T_BODY_2 ""<br>#define T_BODY_3 ""                                                          |
|  |  |  |  | tokenDATA = "DATA" >> STRICT_BOUNDARIES;         |  | #define T_DATA_0 "DATA"<br>#define T_DATA_1 ""<br>#define T_DATA_2 ""<br>#define T_DATA_3 ""                                                          |
|  |  |  |  | tokenBEGIN = "BEGIN" >> STRICT_BOUNDARIES;       |  | #define T_BEGIN_0 "BEGIN"<br>#define T_BEGIN_1 ""<br>#define T_BEGIN_2 ""<br>#define T_BEGIN_3 ""                                                     |
|  |  |  |  | tokenEND = "END" >> STRICT_BOUNDARIES;           |  | #define T_END_0 "END"<br>#define T_END_1 ""<br>#define T_END_2 ""<br>#define T_END_3 ""                                                               |
|  |  |  |  | tokenBEGINBLOCK = "{" >> BOUNDARIES;             |  | #define T_BEGIN_BLOCK_0 "{"<br>#define T_BEGIN_BLOCK_1 ""<br>#define T_BEGIN_BLOCK_2 ""<br>#define T_BEGIN_BLOCK_3 ""                                 |
|  |  |  |  | tokenENDBLOCK = "}" >> BOUNDARIES;               |  | #define T_END_BLOCK_0 "}"<br>#define T_END_BLOCK_1 ""<br>#define T_END_BLOCK_2 ""<br>#define T_END_BLOCK_3 ""                                         |
|  |  |  |  | tokenLEFTSQUAREBRACKETS = "[" >> BOUNDARIES;     |  | #define T_LEFT_SQUAREBRACKETS_0 "["<br>#define T_LEFT_SQUAREBRACKETS_1 ""<br>#define T_LEFT_SQUAREBRACKETS_2 ""<br>#define T_LEFT_SQUAREBRACKETS_3 "" |
|  |  |  |  | tokenRIGHTSQUAREBRACKETS = "]" >> BOUNDARIES;    |  | #define T_RIGHT_SQUAREBRACKETS_0 "]"<br>#define T_RIGHT_SQUAREBRACKETS_1 ""<br>#define T_RIGHT_SQUAREBRACKETS_2 ""                                    |



|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  |                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------|--|--|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | #define T_RIGHT_SQUAREBRACKETS_3 ""                                                                                                           |
|                                                                                                          |  |  |  | tokenSEMICOLON = ";" >> BOUNDARIES;                                                                                                                                                              |  | #define T_SEMICOLON_0 ";"<br>#define T_SEMICOLON_1 ""<br>#define T_SEMICOLON_2 ""<br>#define T_SEMICOLON_3 ""                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | #define T_NULL_STATEMENT_0 "NULL"<br>#define T_NULL_STATEMENT_1 "STATEMENT"<br>#define T_NULL_STATEMENT_2 ""<br>#define T_NULL_STATEMENT_3 "" |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | #define GRAMMAR_LL2__2025 {\                                                                                                                  |
| labeled_point = ident , ":";                                                                             |  |  |  | labeled_point = ident >> tokenCOLON;                                                                                                                                                             |  | { LA_IS, ("ident_terminal"), { "labeled_point", {\                                                                                            |
| goto_label = "GOTO" , ident;                                                                             |  |  |  | goto_label = tokenGOTO >> ident;                                                                                                                                                                 |  | { LA_IS, (T_GOTO_0), { "goto_label", {\                                                                                                       |
| program_name = ident;                                                                                    |  |  |  | program_name = SAME_RULE(ident);                                                                                                                                                                 |  | { LA_IS, ("ident_terminal"), { "program_name", {\                                                                                             |
| value_type = "INTEGER16";                                                                                |  |  |  | value_type = SAME_RULE(tokenINTEGER16);                                                                                                                                                          |  | { LA_IS, (T_DATA_TYPE_0), { "value_type", {\                                                                                                  |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_DATA_TYPE_0}}\                                                                                                           |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, ("["), { "array_specify", {\                                                                                                         |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 3, {"[", "unsigned_value", "]"}}\                                                                                              |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| declaration_element = ident , [ "[", unsigned_value , "]" ];                                             |  |  |  | declaration_element = ident >> -(tokenLEFTSQUAREBRACKETS >> unsigned_value >> tokenRIGHTSQUAREBRACKETS);                                                                                         |  | { LA_IS, ("ident_terminal"), { "declaration_element", {\                                                                                      |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 2, {"ident", "array_specify_optional"}}\                                                                                       |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, ("["), { "array_specify_optional", {\                                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {"array_specify"}}\                                                                                                         |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| other_declaration_ident = " , declaration_element;                                                       |  |  |  | other_declaration_ident = tokenCOMMA >> declaration_element;                                                                                                                                     |  | { LA_NOT, ("["), { "array_specify_optional", {\                                                                                               |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 0, {""}}\                                                                                                                      |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| declaration = value_type , declaration_element , [other_declaration_ident];                              |  |  |  | declaration = value_type >> declaration_element >> *other_declaration_ident;                                                                                                                     |  | { LA_IS, (T_COMA_0), { "other_declaration_ident", {\                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 2, {T_COMA_0, "declaration_element"}}\                                                                                         |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (T_DATA_TYPE_0), { "declaration", {\                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 3, {"value_type", "declaration_element",                                                                                       |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | "other_declaration_ident_iteration"}}\                                                                                                        |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_COMA_0}, { "other_declaration_ident_iteration", {\                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 2, { "other_declaration_ident",                                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | "other_declaration_ident_iteration"}}\                                                                                                        |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_NOT, {T_COMA_0}, { "other_declaration_ident_iteration", {\                                                                               |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 0, {""}}\                                                                                                                      |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| index_action = "[" , expression , "]" ;                                                                  |  |  |  | index_action = tokenLEFTSQUAREBRACKETS >> expression >> tokenRIGHTSQUAREBRACKETS;                                                                                                                |  | { LA_IS, ("["), { "index_action", {\                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 3, {"[", "expression", "]"}}\                                                                                                  |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| unary_operator = "NOT";                                                                                  |  |  |  | unary_operator = SAME_RULE(tokenNOT);                                                                                                                                                            |  | { LA_IS, {T_NOT_0}, { "unary_operator", {\                                                                                                    |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_NOT_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| unary_operation = unary_operator , expression;                                                           |  |  |  | unary_operation = unary_operator >> expression;                                                                                                                                                  |  | { LA_IS, {T_NOT_0}, { "unary_operation", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 2, { "unary_operator", "expression" }}\                                                                                        |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_AND_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_AND_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_OR_0}, { "binary_operator", {\                                                                                                    |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_OR_0}}\                                                                                                                  |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_EQUAL_0}, { "binary_operator", {\                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_EQUAL_0}}\                                                                                                               |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_NOT_EQUAL_0}, { "binary_operator", {\                                                                                             |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_NOT_EQUAL_0}}\                                                                                                           |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | IF_NONUSE_COMPARE_WITH_EQUAL(\                                                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_LESS_0}, { "binary_operator", {\                                                                                                  |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_LESS_0}}\                                                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_GREATER_0}, { "binary_operator", {\                                                                                               |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_GREATER_0}}\                                                                                                             |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | \                                                                                                                                             |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | IF_USE_COMPARE_WITH_EQUAL(\                                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_LESS_OR_EQUAL_0}, { "binary_operator", {\                                                                                         |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_LESS_OR_EQUAL_0}}\                                                                                                       |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_GREATER_OR_EQUAL_0}, { "binary_operator", {\                                                                                      |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_GREATER_OR_EQUAL_0}}\                                                                                                    |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | \                                                                                                                                             |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_ADD_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_ADD_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_SUB_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_SUB_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_MUL_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_MUL_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_DIV_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_DIV_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| binary_operator = "AND"   "OR"   "="   "!="   "<="   ">="   "<"   ">"   "+"   "-"   "*"   "DIV"   "MOD"; |  |  |  | binary_operator = tokenAND   tokenOR   tokenEQUAL   tokenNOTEQUAL   tokenLESSOREQUAL   tokenGREATEROREQUAL   tokenLESS   tokenGREATER   tokenPLUS   tokenMINUS   tokenMUL   tokenDIV   tokenMOD; |  | { LA_IS, {T_MOD_0}, { "binary_operator", {\                                                                                                   |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 1, {T_MOD_0}}\                                                                                                                 |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |
| binary_action = binary_operator , expression;                                                            |  |  |  | binary_action = binary_operator >> expression;                                                                                                                                                   |  | IF_NONUSE_COMPARE_WITH_EQUAL(\                                                                                                                |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, {T_AND_0,T_OR_0,T_EQUAL_0,T_NOT_EQUAL_0,T_LESS_0,                                                                                    |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | T_GREATER_0,T_ADD_0,T_SUB_0,T_MUL_0,T_DIV_0,T_MOD_0}, {                                                                                       |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | "binary_action", {\                                                                                                                           |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | { LA_IS, (""), 2, { "binary_operator", "expression" }}\                                                                                       |
|                                                                                                          |  |  |  |                                                                                                                                                                                                  |  | }};\                                                                                                                                          |

|                                                                                                                      |  |  |  |                                                                                                                                                   |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------|--|--|--|---------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> \ IF_USE_COMPARE_WITH_EQUAL( { LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action", {   { LA_IS, { "" }, 2, { "binary_operator", "expression" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <pre> left_expression = group_expression   unary_operation   ident , [index_action]   value   cond_block; </pre>     |  |  |  | <pre> left_expression = group_expression   unary_operation   ident &gt;&gt; -index_action   value   cond_block; </pre>                            |  | <pre> {LA_IS, { "(" }, { "left_expression", {   {LA_IS, { "" }, 1, { "group_expression" } } } }})\ {LA_IS, { T_NOT_0 }, { "left_expression", {   {LA_IS, { "" }, 1, { "unary_operation" } } } }})\ {LA_IS, { T_ADD_0, T_SUB_0 }, { "left_expression", {   {LA_IS, { "unsigned_value_terminal" }, 1, { "value" } } },   {LA_NOT, { "unsigned_value_terminal" }, 1, { "unary_operation" } } } }})\ {LA_IS, { "ident_terminal" }, { "left_expression", {   {LA_IS, { "" }, 2, { "ident", "index_action__optional" } } } }})\ {LA_IS, { "unsigned_value_terminal" }, { "left_expression", {   {LA_IS, { "" }, 1, { "value" } } } }})\ {LA_IS, { "T_IF_0" }, { "left_expression", {   {LA_IS, { "" }, 1, { "cond_block" } } } }})\ </pre>                                                                                                                                                                                                                                                                |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> {LA_IS, { "[" }, { "index_action__optional", {   {LA_IS, { "" }, 1, { "index_action" } } } }})\ {LA_NOT, { "[" }, { "index_action__optional", {   {LA_IS, { "" }, 0, { "" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <pre> expression = left_expression , {binary_action}; </pre>                                                         |  |  |  | <pre> expression = left_expression &gt;&gt; *binary_action; </pre>                                                                                |  | <pre> {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression", {   {LA_IS, { "" }, 2, { "left_expression", "binary_action__iteration" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> IF_NONUSE_COMPARE_WITH_EQUAL( {LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {   {LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" } } } }})\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_0, T_GREATER_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {   {LA_IS, { "" }, 0, { "" } } } }})\ )\ IF_USE_COMPARE_WITH_EQUAL( {LA_IS, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {   {LA_IS, { "" }, 2, { "binary_action", "binary_action__iteration" } } } }})\ {LA_NOT, { T_AND_0, T_OR_0, T_EQUAL_0, T_NOT_EQUAL_0, T_LESS_OR_EQUAL_0, T_GREATER_OR_EQUAL_0, T_ADD_0, T_SUB_0, T_MUL_0, T_DIV_0, T_MOD_0 }, { "binary_action__iteration", {   {LA_IS, { "" }, 0, { "" } } } }})\ )\ </pre> |
| <pre> group__expression = "(" , expression , ")"; </pre>                                                             |  |  |  | <pre> group_expression = tokenGROUPEXPRESSIONBEGIN &gt;&gt; expression &gt;&gt; tokenGROUPEXPRESSIONEND; </pre>                                   |  | <pre> {LA_IS, { "(" }, { "group_expression", {   {LA_IS, { "" }, 3, { "(" , "expression", ")" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <pre> expression_or_bind_left_to_right = expression , ["=" , ident , [index_action]]; </pre>                         |  |  |  | <pre> expression_or_bind_left_to_right = expression &gt;&gt; -(tokenLRBIND &gt;&gt; ident &gt;&gt; -index_action); </pre>                         |  | <pre> {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "expression_or_bind_left_to_right", {   {LA_IS, { "" }, 2, { "expression", "bind_to_right__optional" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> {LA_IS, { T_LRBIND_0 }, { "bind_to_right", {   {LA_IS, { "" }, 3, { T_LRBIND_0, "ident", "index_action__optional" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> { LA_IS, { T_LRBIND_0 }, { "bind_to_right__optional", {   { LA_IS, { "" }, 1, { "bind_to_right" } } } }})\ { LA_NOT, { T_LRBIND_0 }, { "bind_to_right__optional", {   { LA_IS, { "" }, 0, { "" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <pre> if_expression = expression; </pre>                                                                             |  |  |  | <pre> if_expression = SAME_RULE(expression); </pre>                                                                                               |  | <pre> {LA_IS, { "(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal", "unsigned_value_terminal", "T_IF_0" }, { "if_expression", {   {LA_IS, { "" }, 1, { "expression" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <pre> body_for_true = block_statements_in_while_and_if_body; </pre>                                                  |  |  |  | <pre> body_for_true = SAME_RULE(block_statements_in_while_and_if_body); </pre>                                                                    |  | <pre> {LA_IS, { T_BEGIN_BLOCK_0 }, { "body_for_true", {   {LA_IS, { "" }, 1, { "block_statements_in_while_and_if_body" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <pre> false_cond_block_without_else = "ELSE" , "IF" , if_expression , body_for_true; </pre>                          |  |  |  | <pre> false_cond_block_without_else = tokenELSE &gt;&gt; tokenIF &gt;&gt; if_expression &gt;&gt; body_for_true; </pre>                            |  | <pre> {LA_IS, { T_ELSE_IF_0 }, { "false_cond_block_without_else", {   {LA_IS, { "" }, 4, { T_ELSE_IF_0, T_ELSE_IF_1, "if_expression", "body_for_true" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <pre> body_for_false = "ELSE" , block_statements_in_while_and_if_body; </pre>                                        |  |  |  | <pre> body_for_false = tokenELSE &gt;&gt; block_statements_in_while_and_if_body; </pre>                                                           |  | <pre> {LA_IS, { T_ELSE_BLOCK_0 }, { "body_for_false", {   {LA_IS, { "" }, 2, { T_ELSE_BLOCK_0, "block_statements" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <pre> cond_block = "IF" , if_expression , body_for_true, {false_cond_block_without_else } , [body_for_false]; </pre> |  |  |  | <pre> cond_block = tokenIF &gt;&gt; if_expression &gt;&gt; body_for_true &gt;&gt; *false_cond_block_without_else &gt;&gt; -body_for_false; </pre> |  | <pre> {LA_IS, { T_IF_0 }, { "cond_block", {   {LA_IS, { "" }, 5, { T_IF_0, "if_expression", "body_for_true", "false_cond_block_without_else__iteration", "body_for_false__optional" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> {LA_IS, { T_ELSE_IF_1 }, { "false_cond_block_without_else__iteration", {   {LA_IS, { T_ELSE_IF_1 }, 2, { "false_cond_block_without_else", "false_cond_block_without_else__iteration" } } } }})\ {LA_NOT, { T_ELSE_IF_1 }, 0, { "" } } } }})\ {LA_NOT, { T_ELSE_IF_0 }, { "false_cond_block_without_else__iteration", {   {LA_IS, { "" }, 0, { "" } } } }})\ </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|                                                                                                                      |  |  |  |                                                                                                                                                   |  | <pre> {LA_IS, { T_ELSE_BLOCK_0 }, { "body_for_false__optional", {   {LA_IS, { "" }, 1, { "body_for_false" } } } }})\ {LA_NOT, { T_ELSE_BLOCK_0 }, { "body_for_false__optional", { </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

|                                                                                                       |  |  |  |                                                                                                                                |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------|--|--|--|--------------------------------------------------------------------------------------------------------------------------------|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                       |  |  |  |                                                                                                                                |  | {LA_IS, {""}, 0, { "" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| cond_block__with_optional_bind = cond_block, [tokenLRBIND , ident , [index_action]];                  |  |  |  | cond_block__with_optional_bind = cond_block >> -(tokenLRBIND >> ident >> -index_action);                                       |  | {LA_IS, { T_IF_0 }, { "cond_block__with_optional_bind", {\n{LA_IS, {""}, 2, { "cond_block", "bind_to_right_optional" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| cycle_begin_expression = expression;                                                                  |  |  |  | cycle_begin_expression = SAME_RULE(expression);                                                                                |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "cycle_begin_expression", {\n{LA_IS, {""}, 1, { "expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| cycle_end_expression = expression;                                                                    |  |  |  | cycle_end_expression = SAME_RULE(expression);                                                                                  |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "cycle_end_expression", {\n{LA_IS, {""}, 1, { "expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| cycle_counter = ident;                                                                                |  |  |  | cycle_counter = SAME_RULE(ident);                                                                                              |  | {LA_IS, { "ident_terminal" }, { "cycle_counter", {\n{LA_IS, {""}, 1, { "ident" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| cycle_counter_lr_init = cycle_begin_expression , "=" , cycle_counter;                                 |  |  |  | cycle_counter_lr_init = cycle_begin_expression >> tokenLRBIND >> cycle_counter;                                                |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_lr_init", {\n{LA_IS, {""}, 3, { "cycle_begin_expression", T_LRBIND_0, "cycle_counter" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| cycle_counter_init = cycle_counter_lr_init;                                                           |  |  |  | cycle_counter_init = SAME_RULE(cycle_counter_lr_init);                                                                         |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_init", {\n{LA_IS, {""}, 1, { "cycle_counter_lr_init" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| cycle_counter_last_value = cycle_end_expression;                                                      |  |  |  | cycle_counter_last_value = SAME_RULE(cycle_end_expression);                                                                    |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "cycle_counter_last_value", {\n{LA_IS, {""}, 1, { "cycle_end_expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| cycle_body = "DO" , ({statement}   block_statements);                                                 |  |  |  | cycle_body = tokenDO >> (statement   block_statements);                                                                        |  | {LA_IS, { T_DO_0 }, { "cycle_body", {\n{LA_IS, {""}, 2, { T_DO_0, "statements_or__block_statements" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|                                                                                                       |  |  |  |                                                                                                                                |  | {LA_IS, { T_TO_0 }, { "forto_direction", {\n{LA_IS, {""}, 1, { T_TO_0 }}\n}};\n{\n{LA_IS, { T_DOWNTO_0 }, { "forto_direction", {\n{LA_IS, {""}, 1, { T_DOWNTO_0 }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| forto_cycle = "FOR" , cycle_counter_init , ("TO"   "DOWNT0") , cycle_counter_last_value , cycle_body; |  |  |  | forto_cycle = tokenFOR >> cycle_counter_init >> (tokenTO   tokenDOWNT0) >> cycle_counter_last_value >> cycle_body;             |  | {LA_IS, { T_FOR_0 }, { "forto_cycle", {\n{LA_IS, {""}, 5, { T_FOR_0, "cycle_counter_init", "forto_direction",\n"cycle_counter_last_value", "cycle_body" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|                                                                                                       |  |  |  | continue_while = SAME_RULE(tokenCONTINUE);                                                                                     |  | {LA_IS, { "T_CONTINUE_WHILE_0" }, { "continue_while", {\n{LA_IS, {""}, 1, { "T_CONTINUE_WHILE_0" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|                                                                                                       |  |  |  | break_while = SAME_RULE(tokenBREAK);                                                                                           |  | {LA_IS, { "T_EXIT_WHILE_0" }, { "break_while", {\n{LA_IS, {""}, 1, { "T_EXIT_WHILE_0" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| statement_in_while_and_if_body = statement   "CONTINUE"   "BREAK";                                    |  |  |  | statement_in_while_and_if_body = statement   continue_while   break_while;                                                     |  | {LA_IS, { "ident_terminal", "(" , T_NOT_0, "unsigned_value_terminal",\nT_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,\nT_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, { "statement_in_while_and_if_body", {\n{LA_IS, {""}, 1, { "statement" }}\n}};\n{\n{LA_IS, { T_CONTINUE_WHILE_0 }, { "statement_in_while_and_if_body", {\n{LA_IS, {""}, 1, { "continue_while" }}\n}};\n{\n{LA_IS, { T_EXIT_WHILE_0 }, { "statement_in_while_and_if_body", {\n{LA_IS, {""}, 1, { "break_while" }}\n}};\n                                                                                                                                                                                           |
| block_statements_in_while_and_if_body = "{" , {statement_in_while_and_if_body} , "}" ;                |  |  |  | block_statements_in_while_and_if_body = tokenBEGINBLOCK >> *statement_in_while_and_if_body >> tokenENDBLOCK;                   |  | {LA_IS, { T_BEGIN_BLOCK_0 }, { "block_statements_in_while_and_if_body", {\n{LA_IS, {""}, 3, { T_BEGIN_BLOCK_0,\n"statement_in_while_and_if_body__iteration", T_END_BLOCK_0 }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|                                                                                                       |  |  |  |                                                                                                                                |  | {LA_IS, { "ident_terminal", "(" , T_NOT_0, "unsigned_value_terminal",\nT_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,\nT_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0,\nT_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration", {\n{LA_IS, {""}, 2, { "statement_in_while_and_if_body",\n"statement_in_while_and_if_body__iteration" }}\n}};\n{\n{LA_NOT, { "ident_terminal", "(" , T_NOT_0, "unsigned_value_terminal",\nT_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,\nT_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0, T_CONTINUE_WHILE_0,\nT_EXIT_WHILE_0 }, { "statement_in_while_and_if_body__iteration", {\n{LA_IS, {""}, 0, { "" }}\n}};\n |
| while_cycle_head_expression = expression;                                                             |  |  |  | while_cycle_head_expression = SAME_RULE(expression);                                                                           |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "while_cycle_head_expression", {\n{LA_IS, {""}, 1, { "expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| while_cycle = "WHILE" , while_cycle_head_expression , block_statements_in_while_and_if_body;          |  |  |  | while_cycle = tokenWHILE >> while_cycle_head_expression >> block_statements_in_while_and_if_body;                              |  | {LA_IS, { T_WHILE_0 }, { "while_cycle", {\n{LA_IS, {""}, 3, { T_WHILE_0, "while_cycle_head_expression",\n"block_statements_in_while_and_if_body" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| repeat_until_cycle_cond = expression;                                                                 |  |  |  | repeat_until_cycle_cond = SAME_RULE(expression);                                                                               |  | {LA_IS, { "(" , T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",\n"unsigned_value_terminal", "T_IF_0" }, { "repeat_until_cycle_cond", {\n{LA_IS, {""}, 1, { "expression" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| repeat_until_cycle = "REPEAT" , ({statement}   block_statements) , "UNTIL" , repeat_until_cycle_cond; |  |  |  | repeat_until_cycle = tokenREPEAT >> (*statement   block_statements) >> tokenUNTIL >> repeat_until_cycle_cond;                  |  | {LA_IS, { T_REPEAT_0 }, { "repeat_until_cycle", {\n{LA_IS, {""}, 4, { T_REPEAT_0, "statements_or__block_statements",\nT_UNTIL_0, "repeat_until_cycle_cond" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|                                                                                                       |  |  |  |                                                                                                                                |  | {LA_IS, { "ident_terminal", "(" , T_NOT_0, "unsigned_value_terminal",\nT_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,\nT_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, {\n"statements_or__block_statements", {\n{LA_IS, {""}, 1, { "statement__iteration" }}\n}};\n{\n{LA_IS, { T_BEGIN_BLOCK_0 }, { "statements_or__block_statements", {\n{LA_IS, {""}, 1, { "block_statements" }}\n}};\n                                                                                                                                                                                                                                                                                              |
| input = "GET" , ( ident , [index_action]   "(" , ident , [index_action] , ")" );                      |  |  |  | input = tokenGET >> (ident >> -index_action   tokenGROUPEXPRESSIONBEGIN >> ident >> -index_action >> tokenGROUPEXPRESSIONEND); |  | {LA_IS, { T_INPUT_0 }, { "input_rule", {\n{LA_IS, {""}, 2, { T_INPUT_0, "argument_for_input" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|                                                                                                       |  |  |  |                                                                                                                                |  | {LA_IS, { "ident_terminal" }, { "argument_for_input", {\n{LA_IS, {""}, 2, { "ident", "index_action__optional" }}\n}};\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |



|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  | {LA_IS, {"(", {"argument_for_input", {\n {LA_IS, {""}, 4, {"(", "ident", "index_action__optional", ")"} }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| output = "PUT" , expression;                                                                                                                                                             |  |  |  | output = tokenPUT >> expression;                                                                                                                                                                                            |  | {LA_IS, { T_OUTPUT_0 }, {"output_rule", {\n {LA_IS, {""}, 2, {T_OUTPUT_0, "expression"}}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| statement = expression_or_bind_left_to_right   cond_block__with_optional_bind  <br>forto_cycle   while_cycle   repeat_until_cycle   labeled_point   goto_label   input  <br>output   ";" |  |  |  | statement = expression_or_bind_left_to_right   cond_block__with_optional_bind  <br>forto_cycle   while_cycle   repeat_until_cycle   labeled_point   goto_label   input   output<br>  tokenSEMICOLON;                        |  | {LA_IS, {"ident_terminal" }, {"statement", {\n {LA_IS, { T_COLON_0 }, 1, {"labeled_point"}}\n {LA_NOT, { T_COLON_0 }, 1, {"expression_or_bind_left_to_right"}}\n }}}\n {LA_IS, {"(", T_NOT_0, "unsigned_value_terminal", T_ADD_0, T_SUB_0 }, {<br>"statement", {\n {LA_IS, {""}, 1, {"expression_or_bind_left_to_right"}}\n }}}\n {LA_IS, { T_IF_0 }, {"statement", {\n {LA_IS, {""}, 1, {"cond_block__with_optional_bind"}}\n }}}\n {LA_IS, { T_FOR_0 }, {"statement", {\n {LA_IS, {""}, 1, {"forto_cycle"}}\n }}}\n {LA_IS, { T_WHILE_0 }, {"statement", {\n {LA_IS, {""}, 1, {"while_cycle"}}\n }}}\n {LA_IS, { T_REPEAT_0 }, {"statement", {\n {LA_IS, {""}, 1, {"repeat_until_cycle"}}\n }}}\n {LA_IS, { T_GOTO_0 }, {"statement", {\n {LA_IS, {""}, 1, {"goto_label"}}\n }}}\n {LA_IS, { T_INPUT_0 }, {"statement", {\n {LA_IS, {""}, 1, {"input_rule"}}\n }}}\n {LA_IS, { T_OUTPUT_0 }, {"statement", {\n {LA_IS, {""}, 1, {"output_rule"}}\n }}}\n {LA_IS, { T_SEMICOLON_0 }, {"statement", {\n {LA_IS, {""}, 1, {";"}}\n }}}\n |
|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  | {LA_IS, {"ident_terminal", "(", T_NOT_0, "unsigned_value_terminal",<br>T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,<br>T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, {"statement__iteration", {\n {LA_IS, {""}, 2, {"statement", "statement__iteration" }}\n }}}\n {LA_NOT, {"ident_terminal", "(", T_NOT_0, "unsigned_value_terminal",<br>T_ADD_0, T_SUB_0, T_IF_0, T_FOR_0, T_WHILE_0, T_REPEAT_0, T_GOTO_0,<br>T_INPUT_0, T_OUTPUT_0, T_SEMICOLON_0 }, {"statement__iteration", {\n {LA_IS, {""}, 0, {"" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| block_statements = "{", {statement}, "}"                                                                                                                                                 |  |  |  | block_statements = tokenBEGINBLOCK >> *statement >> tokenENDBLOCK;                                                                                                                                                          |  | {LA_IS, { T_BEGIN_BLOCK_0 }, {"block_statements", {\n {LA_IS, {""}, 3, {T_BEGIN_BLOCK_0, "statement__iteration",<br>T_END_BLOCK_0 }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  | {LA_IS, {"(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",<br>"unsigned_value_terminal", "T_IF_0"}, {"expression__optional", {\n {LA_IS, {""}, 1, {"expression" }}\n }}}\n {LA_NOT, {"(", T_NOT_0, T_ADD_0, T_SUB_0, "ident_terminal",<br>"unsigned_value_terminal", "T_IF_0"}, {"expression__optional", {\n {LA_IS, {""}, 0, {"" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| program = "NAME" , program_name , ",", "BODY" , "DATA" , [declaration] , ";",<br>{statement} , "END";                                                                                    |  |  |  | program = BOUNDARIES >> tokenNAME >> program_name >> tokenSEMICOLON >><br>tokenBODY >> tokenDATA >> (-declaration) >> tokenSEMICOLON >> *statement >><br>tokenEND;                                                          |  | {LA_IS, { T_NAME_0 }, {"program_rule", {\n {LA_IS, {""}, 9, {T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0,<br>T_DATA_0, "declaration__optional", T_SEMICOLON_0, "statement__iteration",<br>T_END_0 }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  | {LA_IS, { T_DATA_TYPE_0 }, {"declaration__optional", {\n {LA_IS, {""}, 1, {"declaration" }}\n }}}\n {LA_NOT, { T_DATA_TYPE_0 }, {"declaration__optional", {\n {LA_IS, {""}, 0, {"" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| digit = "0"   "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9";                                                                                                                       |  |  |  | digit = digit_0   digit_1   digit_2   digit_3   digit_4   digit_5   digit_6   digit_7   digit_8  <br>digit_9;                                                                                                               |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| non_zero_digit = "1"   "2"   "3"   "4"   "5"   "6"   "7"   "8"   "9";                                                                                                                    |  |  |  | non_zero_digit = digit_1   digit_2   digit_3   digit_4   digit_5   digit_6   digit_7   digit_8  <br>digit_9;                                                                                                                |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| unsigned_value = (non_zero_digit , (digit))   "0";                                                                                                                                       |  |  |  | unsigned_value = (non_zero_digit >> *digit   digit_0) >> BOUNDARIES;                                                                                                                                                        |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| value = [sign] , unsigned_value;                                                                                                                                                         |  |  |  | value = -sign >> unsigned_value >> BOUNDARIES;                                                                                                                                                                              |  | {LA_IS, {"unsigned_value_terminal" }, {"unsigned_value", {\n {LA_IS, {""}, 1, {"unsigned_value_terminal" }}\n }}}\n {LA_IS, {"unsigned_value_terminal", T_ADD_0, T_SUB_0 }, {"value", {\n {LA_IS, {""}, 2, {"sign_optional", "unsigned_value" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                                                                                                                                                                                          |  |  |  |                                                                                                                                                                                                                             |  | {LA_IS, { T_ADD_0, T_SUB_0 }, {"sign_optional", {\n {LA_IS, {""}, 1, {"sign" }}\n }}}\n {LA_NOT, { T_ADD_0, T_SUB_0 }, {"sign_optional", {\n {LA_IS, {""}, 0, {"" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| letter_in_lower_case = "a"   "b"   "c"   "d"   "e"   "f"   "g"   "h"   "i"   "j"   "k"   "l"   "m"  <br>"n"   "o"   "p"   "q"   "r"   "s"   "t"   "u"   "v"   "w"   "x"   "y"   "z";     |  |  |  | letter_in_lower_case = a   b   c   d   e   f   g   h   i   j   k   l   m   n   o   p   q   r   s   t   u<br>  v   w   x   y   z;                                                                                            |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| letter_in_upper_case = "A"   "B"   "C"   "D"   "E"   "F"   "G"   "H"   "I"   "J"   "K"   "L"  <br>"M"   "N"   "O"   "P"   "Q"   "R"   "S"   "T"   "U"   "V"   "W"   "X"   "Y"   "Z";     |  |  |  | letter_in_upper_case = A   B   C   D   E   F   G   H   I   J   K   L   M   N   O   P   Q   R   S  <br>T   U   V   W   X   Y   Z;                                                                                            |  | \                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| ident = "._", letter_in_upper_case , letter_in_upper_case , letter_in_upper_case ,<br>letter_in_upper_case , letter_in_upper_case , letter_in_upper_case ,<br>letter_in_upper_case ;     |  |  |  | ident = tokenUNDERSCORE >> letter_in_upper_case >> letter_in_upper_case >><br>letter_in_upper_case >> letter_in_upper_case >> letter_in_upper_case >><br>letter_in_upper_case >> letter_in_upper_case >> STRICT_BOUNDARIES; |  | {LA_IS, {"ident_terminal" }, {"ident", {\n {LA_IS, {""}, 1, {"ident_terminal" }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| sign = "+"   "-";                                                                                                                                                                        |  |  |  | sign = sign_plus   sign_minus;                                                                                                                                                                                              |  | {LA_IS, { T_ADD_0 }, {"sign", {\n {LA_IS, {""}, 1, {"sign_plus" }}\n }}}\n {LA_IS, { T_SUB_0 }, {"sign", {\n {LA_IS, {""}, 1, {"sign_minus" } }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|                                                                                                                                                                                          |  |  |  | sign_plus = SAME_RULE(tokenPLUS);                                                                                                                                                                                           |  | {LA_IS, { T_ADD_0 }, {"sign_plus", {\n {LA_IS, {""}, 1, {T_ADD_0}}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|                                                                                                                                                                                          |  |  |  | sign_minus = SAME_RULE(tokenMINUS);                                                                                                                                                                                         |  | {LA_IS, { T_SUB_0 }, {"sign_minus", {\n {LA_IS, {""}, 1, {T_SUB_0} }}\n }}}\n                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |



[illegible]



|  |  |  |                                                                                                           |  |   |
|--|--|--|-----------------------------------------------------------------------------------------------------------|--|---|
|  |  |  | STRICT_BOUNDARIES = (BOUNDARY >> *(BOUNDARY))   (!qi::alpha   qi::char_("_ "));                           |  | \ |
|  |  |  | BOUNDARIES = (BOUNDARY >> *(BOUNDARY)   NO_BOUNDARY);                                                     |  | \ |
|  |  |  | BOUNDARY = BOUNDARY_SPACE   BOUNDARY_TAB   BOUNDARY_CARRIAGE_RETURN   BOUNDARY_LINE_FEED   BOUNDARY_NULL; |  | \ |
|  |  |  | BOUNDARY_SPACE = " ";                                                                                     |  | \ |
|  |  |  | BOUNDARY_TAB = "\t";                                                                                      |  | \ |
|  |  |  | BOUNDARY_CARRIAGE_RETURN = "\r";                                                                          |  | \ |
|  |  |  | BOUNDARY_LINE_FEED = "\n";                                                                                |  | \ |
|  |  |  | BOUNDARY_NULL = "\0";                                                                                     |  | \ |
|  |  |  | NO_BOUNDARY = "";                                                                                         |  | \ |
|  |  |  | tokenUNDERSCORE = "_";                                                                                    |  | \ |
|  |  |  | A = "A";                                                                                                  |  | \ |
|  |  |  | B = "B";                                                                                                  |  | \ |
|  |  |  | C = "C";                                                                                                  |  | \ |
|  |  |  | D = "D";                                                                                                  |  | \ |
|  |  |  | E = "E";                                                                                                  |  | \ |
|  |  |  | F = "F";                                                                                                  |  | \ |
|  |  |  | G = "G";                                                                                                  |  | \ |
|  |  |  | H = "H";                                                                                                  |  | \ |
|  |  |  | I = "I";                                                                                                  |  | \ |
|  |  |  | J = "J";                                                                                                  |  | \ |
|  |  |  | K = "K";                                                                                                  |  | \ |
|  |  |  | L = "L";                                                                                                  |  | \ |
|  |  |  | M = "M";                                                                                                  |  | \ |
|  |  |  | N = "N";                                                                                                  |  | \ |
|  |  |  | O = "O";                                                                                                  |  | \ |
|  |  |  | P = "P";                                                                                                  |  | \ |
|  |  |  | Q = "Q";                                                                                                  |  | \ |
|  |  |  | R = "R";                                                                                                  |  | \ |
|  |  |  | S = "S";                                                                                                  |  | \ |
|  |  |  | T = "T";                                                                                                  |  | \ |
|  |  |  | U = "U";                                                                                                  |  | \ |
|  |  |  | V = "V";                                                                                                  |  | \ |
|  |  |  | W = "W";                                                                                                  |  | \ |
|  |  |  | X = "X";                                                                                                  |  | \ |
|  |  |  | Y = "Y";                                                                                                  |  | \ |
|  |  |  | Z = "Z";                                                                                                  |  | \ |
|  |  |  | a = "a";                                                                                                  |  | \ |
|  |  |  | b = "b";                                                                                                  |  | \ |
|  |  |  | c = "c";                                                                                                  |  | \ |
|  |  |  | d = "d";                                                                                                  |  | \ |
|  |  |  | e = "e";                                                                                                  |  | \ |
|  |  |  | f = "f";                                                                                                  |  | \ |
|  |  |  | g = "g";                                                                                                  |  | \ |
|  |  |  | h = "h";                                                                                                  |  | \ |
|  |  |  | i = "i";                                                                                                  |  | \ |
|  |  |  | j = "j";                                                                                                  |  | \ |
|  |  |  | k = "k";                                                                                                  |  | \ |
|  |  |  | l = "l";                                                                                                  |  | \ |
|  |  |  | m = "m";                                                                                                  |  | \ |
|  |  |  | n = "n";                                                                                                  |  | \ |
|  |  |  | o = "o";                                                                                                  |  | \ |
|  |  |  | p = "p";                                                                                                  |  | \ |
|  |  |  | q = "q";                                                                                                  |  | \ |
|  |  |  | r = "r";                                                                                                  |  | \ |
|  |  |  | s = "s";                                                                                                  |  | \ |

|  |  |  |  |          |  |                                                                                                                                                                                                                                                                                                |
|--|--|--|--|----------|--|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |  |  |  | t = "t"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | u = "u"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | v = "v"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | w = "w"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | x = "x"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | y = "y"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  | z = "z"; |  | \                                                                                                                                                                                                                                                                                              |
|  |  |  |  |          |  | \         \         \         \         { LA_IS, { T_NAME_0 }, { "program____part1", {\         { LA_IS, { "" }, 7, { T_NAME_0, "program_name", T_SEMICOLON_0, T_BODY_0,\         T_DATA_0, "declaration_optional", T_SEMICOLON_0 }}\         }}}\         \         }\         "program_rule" |